

ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA
PROGRAMA CIENCIAS DE LA COMPUTACIÓN E
INTELIGENCIA ARTIFICIAL
ASIGNATURA LENGUAJES DE PROGRAMACIÓN Y
TRANSDUCCIÓN

compilATOR

Diseño e Implementación de un Analizador
Sintáctico

NASM x86-64

Estudiantes: Laura Vargas
Sebastián Tibata
Juan Riaño

Docente: William Frasser Acevedo

Fecha: Mayo 2026

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	4
3. Tecnologías de Software Utilizadas	4
3.1. NASM (Netwide Assembler)	4
3.2. GNU Linker (ld)	5
3.3. Equivalencia con CUP y JFlex	5
3.4. IDE Utilizado	5
3.5. Librerías Utilizadas	6
4. Diseño del Analizador Sintáctico	6
4.1. Gramática Definida	6
4.2. Reglas de Producción	8
4.2.1. Sentencias	8
4.2.2. Expresiones	8
4.2.3. Condiciones	9
4.3. Manejo de Expresiones	9
4.4. Precedencia y Asociatividad	10
5. Integración entre el Lexer y el Parser	10
6. Implementación del Proyecto	11
6.1. Estructura de Archivos	11
6.2. Generación del Parser	12
6.2.1. Módulo <code>ast.asm</code> — Arena Allocator	12
6.2.2. Módulo <code>strpool.asm</code> — String Pool	13
6.2.3. Módulo <code>parser.asm</code> — Descenso Recursivo	14
7. Pruebas Realizadas	15
7.1. Ejemplo de Entrada	15
7.2. Resultado Esperado	16
7.3. Pruebas de Error	16
7.3.1. Error: identificador sin expresión	16
7.3.2. Error: <code>let</code> sin <code>be</code>	16
7.3.3. Error: lista sin cerrar	17
8. Manejo de Errores Sintácticos	17
9. Dificultades Encontradas	18
9.1. Desincronización entre constantes de tokens	18
9.2. Semántica de <code>advance()</code> y <code>expect()</code>	18
9.3. Expresión <code>path</code> no reconocida como <code>range_expr</code>	18
9.4. LHS de comparaciones restringido a <code>access_expr</code>	18

10.Soluciones Implementadas	19
11.Resultados Obtenidos	19
12.Conclusiones	20
13.Recomendaciones	20
14.Bibliografía	20
15.Anexos	21
15.1. Diseño de Pruebas	21
15.2. Resultados de Pruebas	21

Introducción

El análisis sintáctico es la segunda etapa del proceso de compilación. Recibe como entrada la secuencia de tokens producida por el analizador léxico y verifica que dicha secuencia se ajuste a la gramática formal del lenguaje. Si la estructura es válida, construye una representación interna denominada **Árbol de Sintaxis Abstracta** (AST), que captura la jerarquía semántica del programa y sirve de entrada para las etapas siguientes.

En este proyecto, el analizador sintáctico fue implementado **desde cero en lenguaje ensamblador NASM x86-64**, sin generadores automáticos de parsers. La elección de ensamblador impone restricciones que en lenguajes de alto nivel son transparentes: toda llamada a función, toda estructura de control y toda gestión de memoria debe ser codificada explícitamente.

El lenguaje objetivo es declarativo y orientado a consultas, lo que simplifica la gramática respecto a un lenguaje imperativo de propósito general: no existen bloques anidados, ni estructuras de control complejas, ni declaraciones de tipo. Cada sentencia describe *qué* se quiere obtener; el compilador determina cómo.

Objetivos

Objetivo General

Diseñar e implementar un analizador sintáctico de descenso recursivo en lenguaje ensamblador NASM x86-64 que, a partir del flujo de tokens generado por el analizador léxico, valide la estructura gramatical del lenguaje declarativo y construya un Árbol de Sintaxis Abstracta en memoria para su procesamiento posterior.

Objetivos Específicos

- Definir formalmente la gramática BNF del lenguaje declarativo, garantizando que sea LL(1) para permitir el análisis con un solo token de lookahead.
- Implementar un asignador de nodos de AST basado en una arena estática (*arena allocator*) que evite el uso de llamadas al sistema para gestión de memoria dinámica.
- Implementar un *string pool* estático para preservar copias persistentes de los lexemas una vez que el buffer del lexer los sobrescribe.
- Implementar el parser de descenso recursivo con una función por no-terminal, siguiendo la convención de llamada System V AMD64 ABI.
- Detectar y reportar errores sintácticos con el número de línea correspondiente, terminando el proceso con código de salida 1.

Tecnologías de Software Utilizadas

NASM (Netwide Assembler)

NASM es el ensamblador utilizado en todas las etapas del proyecto. Produce archivos objeto en formato ELF64 para Linux x86-64.

Características utilizadas:

- Directivas `global` y `extern` para la vinculación entre módulos.
- Directiva `%include` para incluir `ast.asm` dentro de `parser.asm` en tiempo de ensamblado.
- Constantes `equ` para los tipos de nodo y offsets del AST.
- Secciones `.data`, `.bss` y `.text` para organizar datos inicializados, datos no inicializados y código respectivamente.

GNU Linker (ld)

Vincula los archivos objeto generados por NASM en un binario ELF64 ejecutable. No se utiliza libc — el binario llama directamente al kernel mediante syscalls.

Equivalencia con CUP y JFlex

La guía de la asignatura describe el uso de **CUP** (Constructor of Useful Parsers) y **JFlex** como herramientas estándar. En este proyecto se optó por una implementación manual en ensamblador. La tabla siguiente muestra la correspondencia funcional:

Concepto	CUP / JFlex	Este proyecto
Lexer	JFlex genera autómatas DFA en Java	<code>lexer.asm</code> : DFA manual en NASM
Parser	CUP genera parser LALR(1) en Java	<code>parser.asm</code> : descenso recursivo LL(1) manual
Tabla de símbolos de tokens	Clase <code>sym.java</code> generada automáticamente	<code>symbol_table.asm</code> : constantes equ
Integración	Interfaz <code>java_cup.runtime.Symbol</code>	Convención de llamada System V AMD64 ABI
Construcción del AST	Acciones semánticas en reglas CUP	Llamadas explícitas a <code>alloc_node</code>

Por qué descenso recursivo en lugar de LALR(1): la gramática del lenguaje declarativo es LL(1) pura — cada producción se selecciona unívocamente con un token de lookahead. Bajo esta condición, el descenso recursivo es equivalente en poder expresivo a LALR(1), más fácil de depurar en ensamblador (una función por no-terminal, flujo lineal), y no requiere las tablas de transición que CUP genera automáticamente.

IDE Utilizado

El desarrollo se realizó en **Sublime Text** sobre Linux (Debian 13), con el repositorio gestionado mediante **Git** y alojado en GitHub: <https://github.com/MrLaction/compilATOR>.

Las pruebas se ejecutan directamente en terminal con `make run <test.src`.

Librerías Utilizadas

No se utilizan librerías externas. El binario es completamente estático y se comunica con el sistema operativo exclusivamente mediante syscalls de Linux:

Syscall	Número	Uso
SYS_READ	0	Leer tokens del lexer (<code>get_token</code>)
SYS_WRITE	1	Imprimir errores a <code>stderr</code>
SYS_EXIT	60	Terminar el proceso

Equivalencia con la librería de CUP:

CUP	Este proyecto
<code>import</code>	Convención de registros: token ID en <code>rax</code> , lexema en <code>lexer_id_name</code>
<code>java_cup.runtime.Symbol;</code>	
<code>Symbol.left</code> (número de línea)	Variable <code>cur_line</code> en <code>.bss</code> del parser

Diseño del Analizador Sintáctico

Gramática Definida

La gramática fue diseñada para ser LL(1): con un token de lookahead, el parser siempre sabe qué producción aplicar. Las decisiones de diseño principales son:

- `where` siempre aplica a la colección más cercana a su izquierda, no al agregado completo. Así, `sum of prices where ...` filtra `prices` antes de agregar.
- `is min` / `is max` dentro de un `where` es una construcción de selección por extremo, distinta del `is` de asignación por su posición dentro de una condición.
- `path`, `next` y `element` son identificadores contextuales, no keywords reservadas. El parser los reconoce comparando el lexema, no el tipo de token.

La gramática completa en notación BNF:

```

program      ::= statement* TK_EOF

statement    ::= assignment TK_NEWLINE
               | let_binding TK_NEWLINE

assignment   ::= TK_ID TK_IS expr

let_binding  ::= TK_LET TK_ID TK_BE expr

expr         ::= aggregate_expr
               | range_expr
               | arith_expr filter_clause?

```

```
aggregate_expr ::= agg_op TK_OF access_expr filter_clause?
agg_op         ::= TK_SUM | TK_MIN | TK_MAX

range_expr     ::= 'path' TK_FROM TK_ID TK_TO TK_ID
                filter_clause?

filter_clause  ::= TK_WHERE condition

condition      ::= cond_term (TK_OR cond_term)*

cond_term      ::= cond_factor (TK_AND cond_factor)*

cond_factor    ::= TK_NOT cond_factor
                | TK EVERY 'element' TK_LESS_EQ 'next'
                | arith_expr TK_IN list_literal
                | arith_expr TK_IS agg_op
                | comparison

comparison     ::= arith_expr relop arith_expr

relop          ::= TK_EQUAL | TK_NEQ | TK_LESS | TK_GREATER
                | TK_LESS_EQ | TK_GREATER_EQ

arith_expr     ::= term ((TK_PLUS | TK_MINUS) term)*

term           ::= factor ((TK_MULTIPLY | TK_DIVIDE |
                TK_MODULO) factor)*

factor         ::= TK_LPAREN arith_expr TK_RPAREN
                | TK_MINUS factor
                | TK_LIT_INT | TK_LIT_FLOAT | TK_LIT_STR
                | TK_TRUE | TK_FALSE
                | access_expr

access_expr    ::= TK_ID (TK_DOT TK_ID)*

list_literal   ::= TK_LBRACKET factor (TK_COMMA factor)*
                TK_RBRACKET
```


Reglas de Producción

Las reglas de producción se implementan como funciones en `parser.asm`. Cada no-terminal corresponde exactamente a una función; el flujo de control entre funciones refleja la estructura de la gramática.

Sentencias

```
statement ::= assignment TK_NEWLINE
           | let_binding TK_NEWLINE

assignment ::= TK_ID TK_IS expr
           -- ejemplo: result is users where age > 18

let_binding ::= TK_LET TK_ID TK_BE expr
           -- ejemplo: let shortest_route be path from A
           to B
```

Las reglas permiten dos formas de declaración: asignación directa (`id IS expr`) y ligadura nombrada (`LET id BE expr`). Ambas generan nodos distintos en el AST (`NODE_ASSIGN` y `NODE_LET` respectivamente) pero comparten la misma función `parse_expr` para el lado derecho.

Expresiones

```
expr ::= aggregate_expr      -- sum of prices where ...
      | range_expr          -- path from A to B where
      | ...
      | arith_expr filter?  -- users where age > 18

aggregate_expr ::= agg_op TK_OF access_expr filter_clause?
agg_op         ::= TK_SUM | TK_MIN | TK_MAX
```

La selección entre las tres alternativas se resuelve con lookahead de 1 token:

- Si `cur_token` es `TK_SUM`, `TK_MIN` o `TK_MAX` → `aggregate_expr`.
- Si `cur_token` es `TK_ID` con lexema "path" → `range_expr`.
- En cualquier otro caso → `arith_expr` con `filter_clause` opcional.

Condiciones

```
condition ::= cond_term (TK_OR cond_term)*
cond_term ::= cond_factor (TK_AND cond_factor)*
cond_factor ::= TK_NOT cond_factor
              | TK_EVERY 'element' TK_LESS_EQ 'next'
              | arith_expr TK_IN list_literal
              | arith_expr TK_IS agg_op
              | comparison
```

La jerarquía `condition` $\rightarrow$ `cond_term` $\rightarrow$ `cond_factor` codifica la precedencia de los operadores lógicos: `and` tiene mayor precedencia que `or`, de manera análoga a como la multiplicación tiene mayor precedencia que la suma en las expresiones aritméticas.

Manejo de Expresiones

Las expresiones aritméticas siguen la jerarquía clásica de precedencia:

```
arith_expr ::= term ((TK_PLUS | TK_MINUS) term)*
term       ::= factor ((TK_MULTIPLY | TK_DIVIDE | TK_MODULO
) factor)*
factor     ::= TK_LPAREN arith_expr TK_RPAREN
              | TK_MINUS factor
              | TK_LIT_INT | TK_LIT_FLOAT | TK_LIT_STR
              | TK_TRUE | TK_FALSE
              | access_expr
```

La gramática reconoce expresiones matemáticas con todos los operadores aritméticos. La recursión en `factor` maneja el menos unario y las expresiones parentizadas con profundidad arbitraria.

Ejemplo de expresión en el lenguaje:

```
tax_total is sum of orders.amount where price + tax >100.0
```

En este caso, `price + tax` es una `arith_expr` que forma el lado izquierdo de una `comparison`.

Precedencia y Asociatividad

La precedencia se implementa implícitamente a través de la jerarquía de no-terminales, sin necesidad de tablas explícitas como en LALR(1):

Nivel	Operadores	Asociatividad
1 (menor)	<code>or</code>	izquierda
2	<code>and</code>	izquierda
3	<code>not</code>	derecha (prefijo)
4	<code>== != <> <= >=</code>	no asociativo
5	<code>+ -</code>	izquierda
6	<code>* /%</code>	izquierda
7 (mayor)	unario <code>-</code> , <code>()</code>	derecha

Los operadores de mayor nivel se evalúan primero porque están más profundos en la jerarquía de no-terminales. La asociatividad izquierda se obtiene naturalmente del loop iterativo en cada nivel (en lugar de recursión a la derecha).

Integración entre el Lexer y el Parser

En CUP/JFlex, el lexer implementa la interfaz `java_cup.runtime.Scanner` y el parser llama a `next_token()` en cada paso. En este proyecto el mecanismo es equivalente pero a nivel de registros y convención de llamada:

CUP / JFlex	Este proyecto
<code>Scanner.next_token()</code>	call <code>get_token</code>
<code>Symbol.sym</code> (tipo de token)	<code>rax</code> al retornar de <code>get_token</code>
<code>Symbol.value</code> (lexema)	<code>lexeme[]</code> (buffer global del lexer)
<code>Symbol.left</code> (número de línea)	<code>cur_line</code> (contador del parser)

La función `advance()` del parser encapsula esta integración:

```

1  advance:
2      push rbp
3      mov  rbp, rsp
4      push rbx
5
6      call get_token          ; rax = token type ID
7      mov  [cur_token], rax
8
9      cmp  rax, TK_NEWLINE    ; incrementar contador de
                                lineas
10     jne  .no_newline
11     mov  rbx, [cur_line]
12     inc  rbx
13     mov  [cur_line], rbx
14 .no_newline:

```

```
15
16     lea    rdi, [lexeme]           ; copiar lexema al string pool
17     call  intern_str              ; rax = puntero estable al
    lexema
18     mov    [cur_lexeme], rax
19
20     pop    rbx
21     pop    rbp
22     ret
```

Cada llamada a `advance()` realiza tres acciones: obtiene el siguiente token del lexer, actualiza el contador de líneas si corresponde, e interna el lexema en el string pool para darle un puntero estable que los nodos del AST puedan almacenar.

Implementación del Proyecto

Estructura de Archivos

```
compilATOR/
|-- lexer/
|   |-- symbol_table.asm    <- constantes de tokens,
|   keywords, tabla de punteros
|   |-- lexer.asm          <- DFA: tokenizador, I/O con
|   buffer
|   |-- main.asm           <- entry point del binario lexer
|   standalone
|   |-- Makefile
|   '-- test.src
|-- parser/
|   |-- ast.asm            <- arena allocator: 2048 nodos x
|   40 bytes
|   |-- strpool.asm        <- string pool: 64 KB para
|   lexemas persistentes
|   |-- parser.asm         <- parser LL(1) de descenso
|   recursivo
|   |-- main.asm           <- entry point del binario
|   parser standalone
|   |-- grammar.md         <- gramatica BNF formal
|   |-- Makefile
|   '-- test.src
'-- README.md
```

Generación del Parser

A diferencia de CUP, que genera automáticamente `parser.java` y `sym.java` a partir de un archivo `.cup`, en este proyecto el parser es escrito a mano. El proceso de construcción es:

```
# Ensamblar módulos del lexer
nasm -f elf64 ../lexer/symbol_table.asm -o symbol_table.o
nasm -f elf64 ../lexer/lexer.asm -o lexer.o

# Ensamblar módulos del parser
# parser.asm incluye ast.asm en tiempo de ensamblado via %
include
nasm -f elf64 strpool.asm -o strpool.o
nasm -f elf64 parser.asm -o parser.o
nasm -f elf64 main.asm -o main.o

# Vincular en un solo binario
ld symbol_table.o lexer.o strpool.o parser.o main.o -o
  parser

# O simplemente:
make run < test.src
```

Los archivos generados y su función:

Archivo	Función
<code>symbol_table.o</code>	Datos estáticos: tokens, keywords, tabla de nombres
<code>lexer.o</code>	DFA tokenizador, función <code>get_token</code>
<code>strpool.o</code>	Pool de strings, función <code>intern_str</code>
<code>parser.o</code>	Parser + arena de nodos AST (<code>ast.asm</code> incluido)
<code>main.o</code>	Entry point <code>_start</code> , llama <code>parse_program</code>
<code>parser</code>	Binario ELF64 ejecutable final

Módulo `ast.asm` — Arena Allocator

Cada nodo del AST tiene una estructura fija de 40 bytes:

Offset	Campo	Propósito
+0	<code>type</code>	Tipo de nodo (constante <code>NODE_*</code>)
+8	<code>left</code>	Puntero al hijo izquierdo
+16	<code>right</code>	Puntero al hijo derecho
+24	<code>value</code>	Valor literal, ID de token, o puntero a string
+32	<code>line</code>	Número de línea fuente (para errores)

La arena reserva 2048 nodos en `.bss` (80 KB). `alloc_node(type, line)` es un simple incremento de contador — sin syscalls, sin fragmentación:

```

1  alloc_node:                ; rdi = type, rsi = line -> rax =
   node pointer
2  mov  rax, [ast_count]
3  cmp  rax, AST_POOL_CAP    ; 2048
4  jge  .pool_full           ; exit code 99 si se agota
5
6  imul rax, rax, NODE_SIZE   ; offset = index * 40
7  lea  rcx, [ast_pool]
8  add  rax, rcx              ; puntero al nodo
9
10     ; inicializar los 5 campos a cero
11  mov  qword [rax + NODE_TYPE], 0
12  mov  qword [rax + NODE_LEFT], 0
13  mov  qword [rax + NODE_RIGHT], 0
14  mov  qword [rax + NODE_VALUE], 0
15  mov  qword [rax + NODE_LINE], 0
16
17  mov  qword [rax + NODE_TYPE], rbx    ; type
18  mov  qword [rax + NODE_LINE], r12    ; line
19
20  mov  rcx, [ast_count]
21  inc  rcx
22  mov  [ast_count], rcx
23  ret

```

Módulo strpool.asm — String Pool

El lexer reutiliza su buffer `lexeme[]` en cada llamada a `get_token`. Los nodos del AST necesitan punteros estables a los nombres de los identificadores. `intern_str(src)` copia el string al pool y devuelve un puntero permanente:

```

1  intern_str:                ; rdi = src -> rax = puntero en pool
2  ; calcular longitud de src
3  ; verificar capacidad: pos + len + 1 <= 65536
4  ; copiar bytes al pool
5  ; escribir null terminator
6  ; avanzar str_pool_pos en len + 1
7  ; retornar puntero al inicio de la copia

```

Módulo `parser.asm` — Descenso Recursivo

El parser implementa una función por no-terminal. El estado global del análisis vive en tres variables en `.bss`:

```
1 section .bss
2     cur_token    resq 1    ; tipo del token actual
3     cur_lexeme   resq 1    ; puntero al lexema internado
4     cur_line     resq 1    ; línea actual del fuente
```

La convención interna del parser es: al entrar a cualquier función, `cur_token` ya contiene el primer token a procesar. Al retornar, `cur_token` contiene el primer token que *no* fue consumido.

Ejemplo de implementación de `parse_assignment`:

```
1 parse_assignment:                ; -> rax = NODE_ASSIGN
2     push rbp
3     mov rbp, rsp
4     push r12
5     push r13
6
7     ; cur_token == TK_ID (verificado por el caller)
8     mov rdi, NODE_ID
9     mov rsi, [cur_line]
10    call alloc_node
11    mov r12, rax                ; r12 = nodo ID
12
13    mov rax, [cur_lexeme]
14    mov qword [r12 + NODE_VALUE], rax ; guardar nombre
    antes de advance
15
16    call advance                ; consumir TK_ID ->
    cur_token = TK_IS
17    mov rdi, TK_IS
18    call expect                ; verificar sin
    avanzar
19
20    call advance                ; consumir TK_IS ->
    cur_token = primer token de expr
21    call parse_expr            ; rax = nodo de la
    expresion
22    mov r13, rax
23
24    mov rdi, NODE_ASSIGN
25    mov rsi, [cur_line]
26    call alloc_node
27    mov qword [rax + NODE_LEFT], r12
28    mov qword [rax + NODE_RIGHT], r13
29
```

```
30     pop    r13
31     pop    r12
32     pop    rbp
33     ret
```

Pruebas Realizadas

Ejemplo de Entrada

El archivo `test.src` contiene los ocho patrones sintácticos del lenguaje, uno por sentencia:

```
-- Filter users older than 18
result is users where age > 18

-- Filter with multiple conditions
active_users is users where age > 18 and active == true

-- Check ordering
sorted is numbers where every element <= next

-- Aggregate with condition
total is sum of prices where category == "food"

-- Find shortest path
let shortest_route be path from A to B where distance is
  min

-- Membership test with list
filtered is orders where user.region in ["north", "south"]

-- Arithmetic in filter
tax_total is sum of orders.amount where price + tax > 100.0

-- Find maximum
let best_score be results where score is max
```


Resultado Esperado

```
$ make run < test.src
nasm -f elf64 ../lexer/symbol_table.asm -o symbol_table.o
nasm -f elf64 ../lexer/lexer.asm -o lexer.o
nasm -f elf64 strpool.asm -o strpool.o
nasm -f elf64 parser.asm -o parser.o
nasm -f elf64 main.asm -o main.o
ld symbol_table.o lexer.o strpool.o parser.o main.o -o
    parser
./parser < test.src
syntax OK
```

El parser acepta los ocho patrones sin errores y termina con código de salida 0.

Pruebas de Error

Error: identificador sin expresión

```
result is
```

Resultado esperado:

```
parse error: unexpected token on line 1
```

El parser espera una expresión después de `is` y encuentra `TK_NEWLINE` en su lugar.

Error: let sin be

```
let x is 42
```

Resultado esperado:

```
parse error: unexpected token on line 1
```

La producción `let_binding` exige `TK_BE` después del identificador. Encontrar `TK_IS` es un error sintáctico.

Error: lista sin cerrar

```
result is items where region in ["north", "south"
```

Resultado esperado:

```
parse error: unexpected token on line 1
```

El parser espera `TK_RBRACKET` o `TK_COMMA` al iterar la lista y encuentra `TK_NEWLINE`.

Manejo de Errores Sintácticos

El parser detecta errores en dos situaciones:

1. **Token inesperado:** `expect(type)` verifica que `cur_token == type`. Si no coincide, llama a `syntax_error`.
2. **Contexto inválido:** una función de no-terminal recibe un token que no inicia ninguna producción válida (por ejemplo, `parse_factor` recibe `TK_WHERE`).

La función `syntax_error` es análoga al método `syntax_error` de CUP:

CUP	Este proyecto
<pre>public void syntax_error(Symbol s) { System.out.println("Error en línea: " + s.left); }</pre>	<pre>syntax_error: ; escribir mensaje a stderr mov rdi, STDERR lea rsi, [err_unexpected] mov rdx, 38 mov rax, SYS_WRITE syscall ; imprimir numero de línea mov rdi, [cur_line] call print_uint ; terminar con exit code 1 mov rax, SYS_EXIT mov rdi, 1 syscall</pre>

El parser **no implementa recuperación de errores** — al detectar el primer error sintáctico, imprime el número de línea y termina el proceso. Esta es la estrategia más simple y la correcta para un compilador de una pasada.

Dificultades Encontradas

Desincronización entre constantes de tokens

El problema más crítico del desarrollo fue que las constantes de token en `parser.asm` no coincidían con las de `symbol_table.asm`. Por ejemplo, `TK_GREATER` era 31 en el parser pero 22 en el lexer. Esto hacía que cada comparación de tipo de token fuera incorrecta, pero el código compilaba sin errores porque NASM no puede verificar la coherencia entre módulos.

El error se manifestó como fallos en tiempo de ejecución para cualquier entrada que contuviera operadores relacionales. La depuración requirió escribir un programa auxiliar que imprimiera los IDs numéricos de los tokens emitidos por el lexer, lo que permitió identificar la discrepancia.

Semántica de `advance()` y `expect()`

La definición inicial de `expect()` llamaba `advance()` internamente, lo que generaba un consumo doble de tokens en todos los sitios de llamada. El parser avanzaba un token de más en cada verificación, saltando el primer token de la expresión siguiente.

La solución fue redefinir `expect()` para que solo verifique `cur_token` sin avanzar. El caller gestiona todos los avances explícitamente, haciendo el flujo predecible y depurable.

Expresión `path` no reconocida como `range_expr`

El keyword contextual `path` no es un token reservado — el lexer lo emite como `TK_ID`. La función `parse_expr` no tenía el chequeo de lexema necesario para derivar a `parse_range_expr`, por lo que `let shortest_route be path from A to B` fallaba al intentar parsear `from` como parte de una expresión aritmética.

Se añadió una comparación de cadena en `parse_expr`: cuando `cur_token == TK_ID` y `cur_lexeme` apunta a "path", se invoca `parse_range_expr`.

LHS de comparaciones restringido a `access_expr`

El caso de prueba `price + tax > 100.0` fallaba porque `parse_cond_factor` usaba `parse_access_expr` para el lado izquierdo de una comparación. Un `access_expr` solo reconoce identificadores con notación de punto, no expresiones aritméticas.

Se reemplazó por `parse_arith_expr`, que cubre el caso degenerado de un solo identificador y además soporta expresiones compuestas.

Soluciones Implementadas

Problema	Solución
Constantes de token inconsistentes	Se reemplazó la tabla de constantes en <code>parser.asm</code> copiando los valores exactos de <code>symbol_table.asm</code> . Se añadió el comentario <code>; must match symbol_table.asm</code> para prevenir futuras discrepancias.
<code>expect()</code> con avance doble	Se redefinió <code>expect()</code> para solo verificar sin avanzar. Todos los callers gestionan <code>advance()</code> explícitamente.
<code>path</code> no detectado como <code>range_expr</code>	Se añadió chequeo de lexema en <code>parse_expr</code> mediante <code>str_eq(cur_lexeme, "path")</code> antes de derivar a <code>parse_range_expr</code> .
LHS aritmético en comparaciones	Se reemplazó <code>parse_access_expr</code> por <code>parse_arith_expr</code> en <code>parse_cond_factor</code> para el LHS de toda comparación.
<code>parse_expr_base</code> indefinida	Se implementó la función faltante <code>parse_arith_expr</code> y se corrigió la referencia en <code>parse_expr</code> .
<code>aggregate_expr</code> rechazaba <code>access_expr</code>	Se reemplazó la verificación manual de <code>TK_ID</code> por una llamada a <code>parse_access_expr</code> , soportando <code>orders.amount</code> .

Resultados Obtenidos

El analizador sintáctico acepta correctamente los ocho patrones sintácticos definidos en el lenguaje:

#	Sentencia	Resultado
1	<code>result is users where age >18</code>	✓
2	<code>active_users is users where age >18 and active == true</code>	✓
3	<code>sorted is numbers where every element <= next</code>	✓
4	<code>total is sum of prices where category == "food"</code>	✓
5	<code>let shortest_route be path from A to B where distance is min</code>	✓
6	<code>filtered is orders where user.region in ["north", "south"]</code>	✓
7	<code>tax_total is sum of orders.amount where price + tax >100.0</code>	✓
8	<code>let best_score be results where score is max</code>	✓

El binario produce `syntax OK` y termina con código de salida 0 para entradas válidas, y `parse error: unexpected token on line N` con código de salida 1 para

entradas inválidas.

El AST se construye en una arena de 2048 nodos de 40 bytes cada uno (80 KB en `.bss`), sin ninguna llamada al sistema para gestión de memoria.

Conclusiones

La implementación del analizador sintáctico en NASM x86-64 demuestra que un parser de descenso recursivo LL(1) puede construirse directamente en ensamblador siguiendo una correspondencia directa entre la gramática formal y el código: cada no-terminal es una función, cada alternativa de producción es una rama condicional.

La ausencia de un generador automático como CUP obligó a resolver explícitamente problemas que herramientas como CUP encapsulan: la sincronización de constantes entre módulos, la semántica precisa del cursor de tokens, y la integración manual entre el lexer y el parser.

La decisión de usar una arena estática de nodos en lugar de asignación dinámica resultó acertada: simplifica el código, elimina toda posibilidad de fragmentación y hace la gestión de memoria completamente predecible.

Recomendaciones

- Centralizar todas las constantes de tokens en un único archivo incluido por todos los módulos (`%include "tokens.asm"`) para evitar desincronizaciones futuras.
- Agregar un modo de depuración que imprima el AST completo en formato indentado, facilitando la verificación visual antes de implementar el análisis semántico.
- Implementar recuperación de errores sintácticos para reportar múltiples errores en una sola ejecución, en lugar de detenerse al primero.

Bibliografía

- Aho, A., Lam, M., Sethi, R., Ullman, J. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
- Intel Corporation. (2023). *Intel 64 and IA-32 Architectures Software Developer's Manual*.
- System V Application Binary Interface — AMD64 Architecture Processor Supplement (2023).
- NASM Development Team. (2023). *NASM — The Netwide Assembler* (versión 2.16). <https://www.nasm.us/doc/>
- Repositorio del proyecto: <https://github.com/MrLaction/compilATOR>

Anexos

Diseño de Pruebas

ID	Entrada	Propósito	Esperado
T01	result is users where age >18	Filtro simple con relop	syntax OK
T02	active_users is users where age >18 and active == true	Condición compuesta con AND	syntax OK
T03	sorted is numbers where every element <= next	Cláusula EVERY	syntax OK
T04	total is sum of prices where category == "food"	Agregación con filtro de string	syntax OK
T05	let shortest_route be path from A to B where distance is min	Range expr + IS EXTREME	syntax OK
T06	filtered is orders where user.region in ["north", "south"]	Acceso con punto + IN + lista	syntax OK
T07	tax_total is sum of orders.amount where price + tax >100.0	Arith expr en filtro + access path	syntax OK
T08	let best_score be results where score is max	IS MAX	syntax OK
T09	result is	Expr faltante	error línea 1
T10	let x is 42	BE faltante	error línea 1
T11	x in [.a", "b"	Lista sin cerrar	error línea 1

Resultados de Pruebas

```
$ ./parser < test.src
syntax OK

$ echo "result is" | ./parser
parse error: unexpected token on line 1

$ echo "let x is 42" | ./parser
parse error: unexpected token on line 1

$ printf 'x in [.a", "b"' | ./parser
parse error: unexpected token on line 1
```

Todas las pruebas producen el resultado esperado.